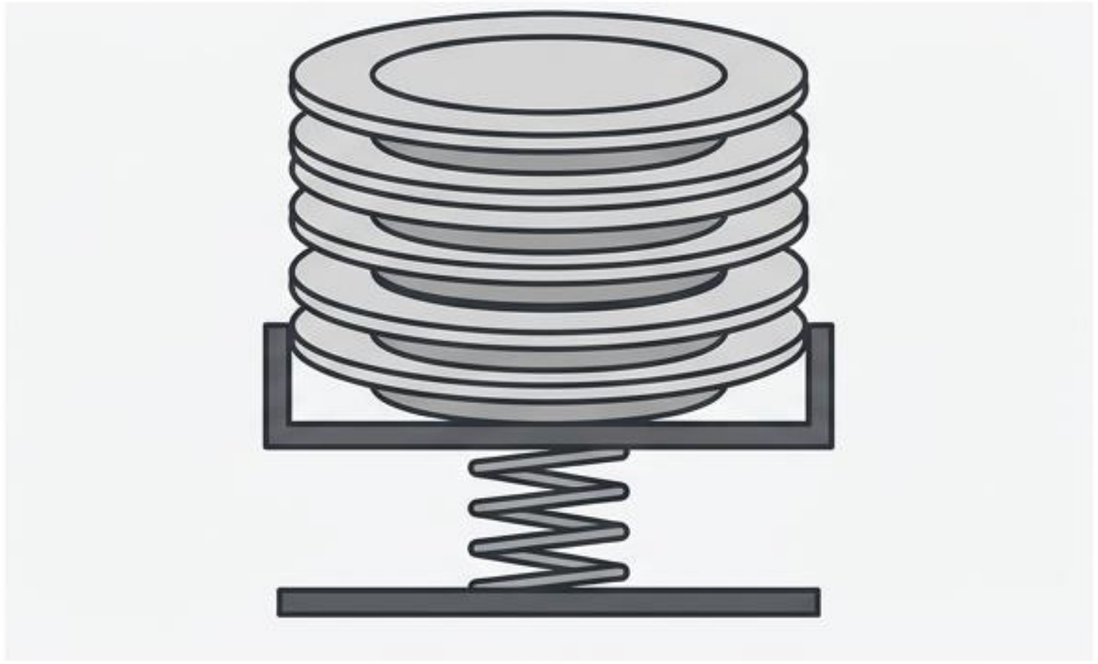
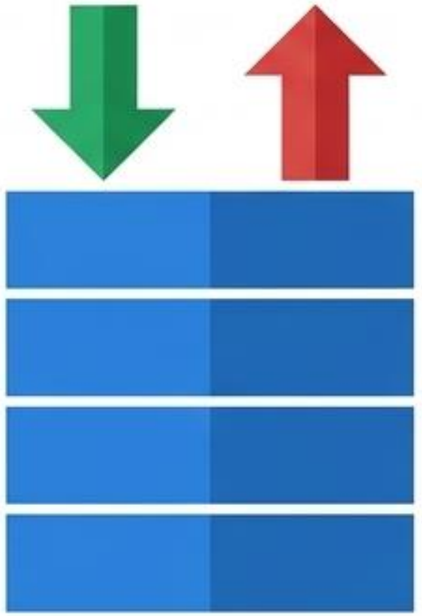
The Stack Data Structure

LIFO Architecture, Array Implementation, and Algorithmic Complexity

The Core Philosophy: Last In, First Out (LIFO)

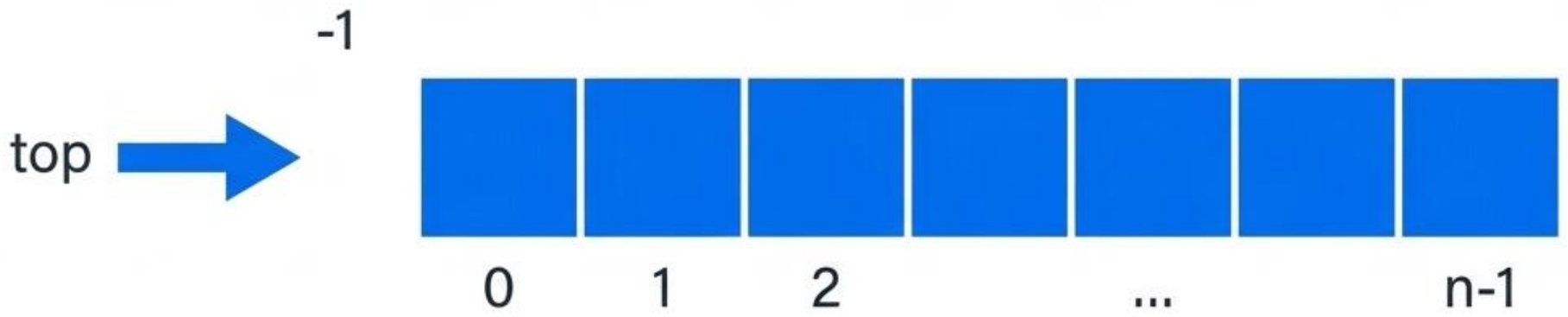




Definition: A dynamic set where the element removed is always the one most recently inserted.

The Golden Rule: Insertions and deletions are strictly confined to a single access point—the top of the stack.

Anatomy of an Array-Based Stack



Storage Framework: A 1-dimensional array Stack[0...n-1] with capacity n.

The State Tracker: A dedicated integer variable top records the precise index of the most recently inserted element.

Initialization: Setting top = -1 definitively indicates an entirely empty stack.

Structural Constraints: Underflow and Overflow

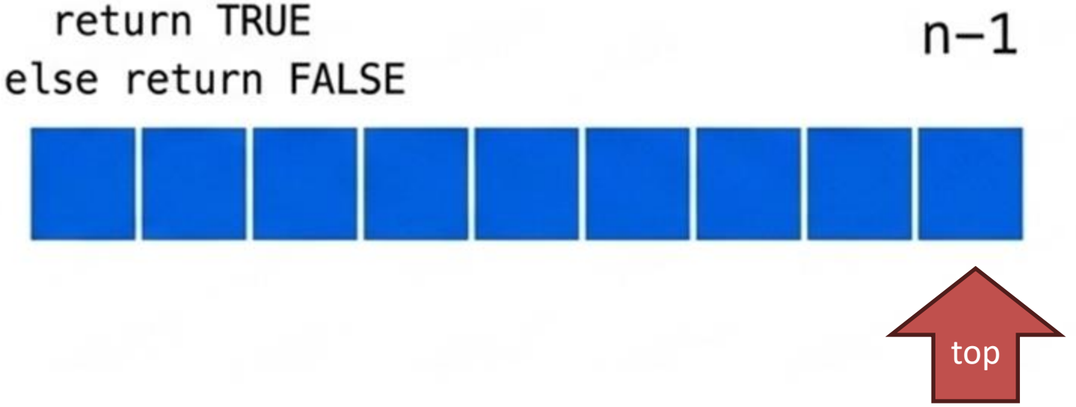
```
STACK-EMPTY(S)
if S.top == -1
  return TRUE
else return FALSE
```



Stack-Empty Condition: `top == -1`

Attempting a deletion from this state triggers an Underflow error. There is no data to retrieve.

```
STACK-FULL(S)
if S.top == n-1
  return TRUE
else return FALSE
```



Stack-Full Condition: `top == n - 1`

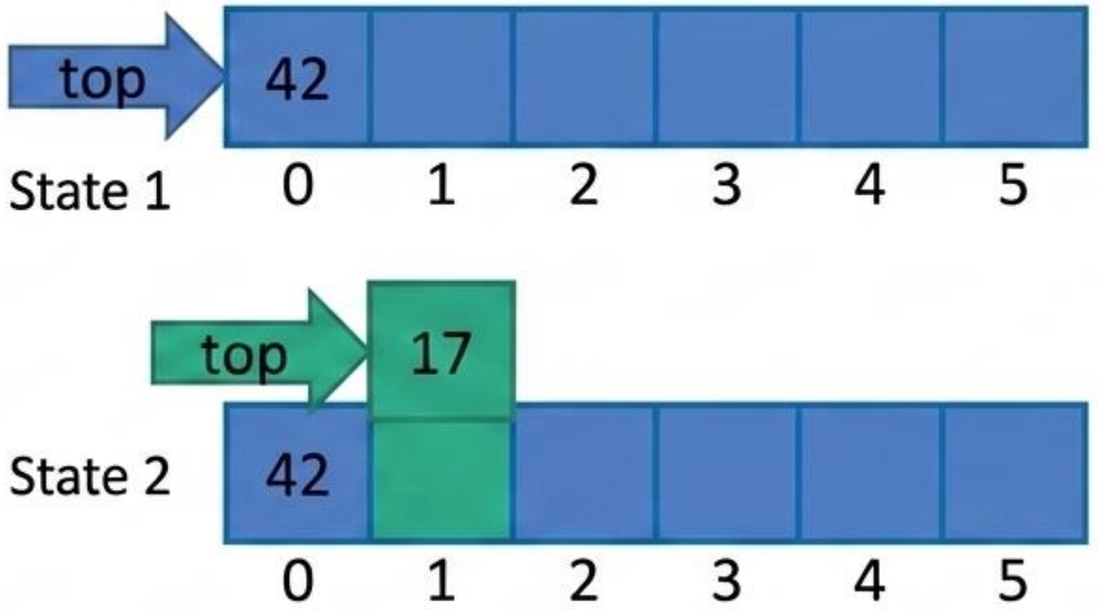
Attempting an insertion into this state triggers an Overflow error. Memory capacity is exhausted.

The PUSH Operation: Inserting Data

Algorithm

```
PUSH (S, x)
  if (STACK_FULL(S)) error “overflow”
  else{
    S.top=S.top+1
    S[S.top]=x
  }
```

Code-to-State Trace



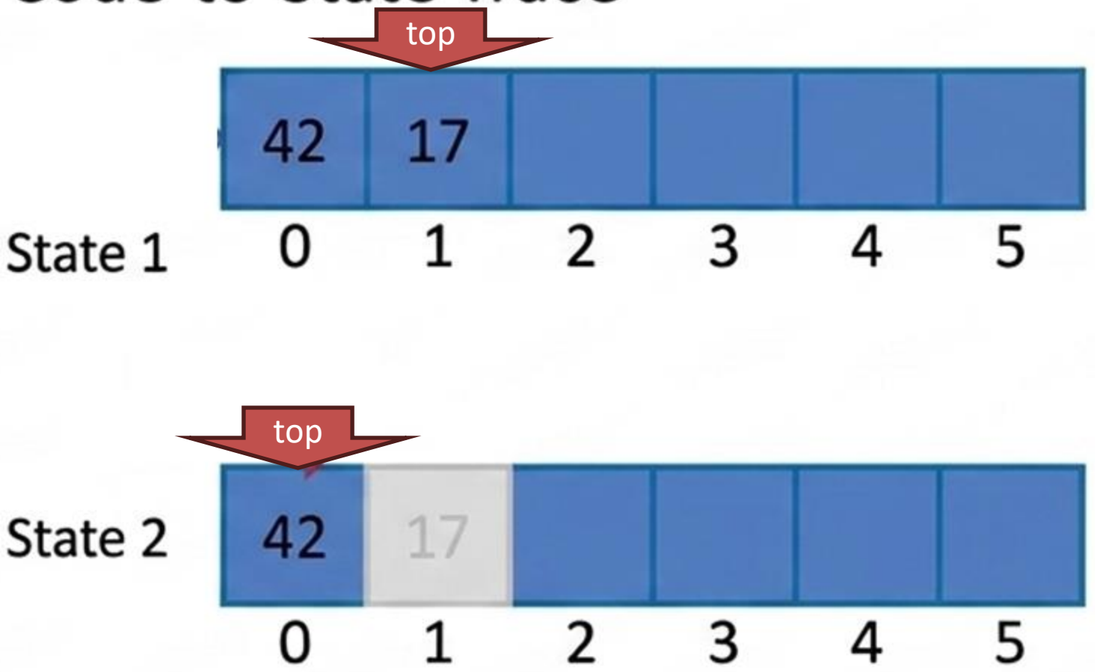
Action: Push(17) onto a stack currently holding 42.

The POP Operation: Extracting Data

Algorithm

```
POP (S)
   if STACK-EMPTY (S) error “underflow”
   else {
        x=S[top]
        S.top =S.top -1
        return x
}
```

Code-to-State Trace



Action: Pop() extracts 17, reducing the active stack back to [42].

Operational Synthesis: Push vs. Pop

Characteristic	Push(x)	Pop()
Pointer Action	Increments (`top`++)	Decrements (`top`−−)
Data Flow	Input to Stack Memory	Output to Target Variable x
Boundary Threat	**Overflow (Stack-Full)**	**Underflow (Stack-Empty)**
Execution Order	Update top, then Write Data	Read Data, then Update top

Algorithmic Complexity

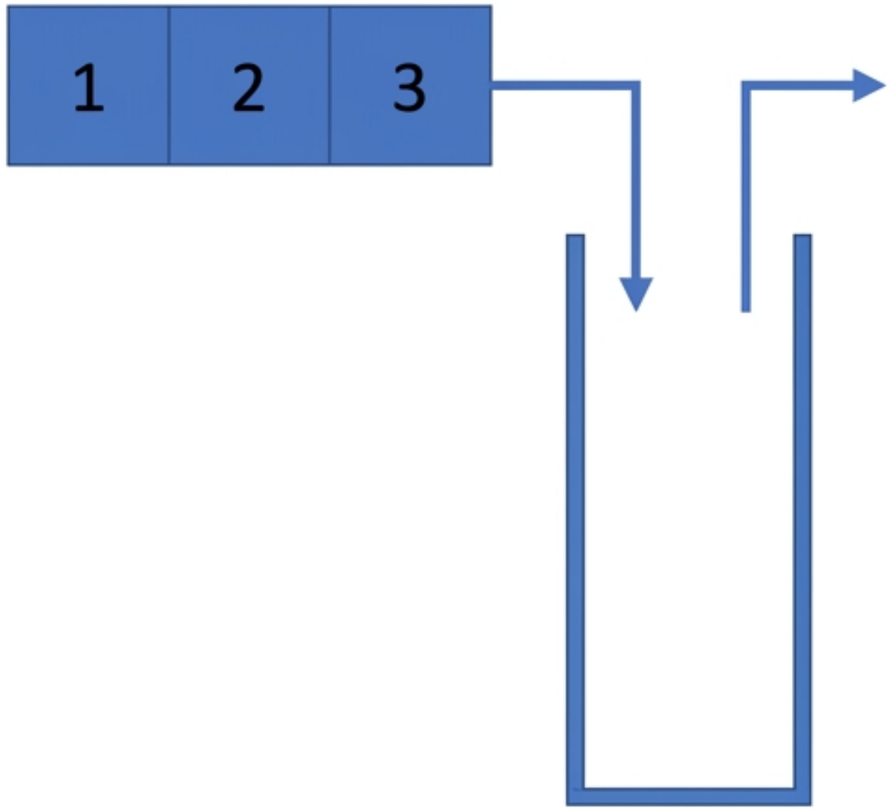
$$O(1)$$

The Notation: Constant Time Execution

Both Push and Pop operations execute in exactly O(1) time, entirely independent of the stack's total capacity (n).

The Proof: The operations rely exclusively on basic arithmetic (+1, -1) applied to a single variable (top) and direct memory access via that index. There are zero loops, zero data shifting, and zero search routines required.

Stack Permutations: The Output Sequence Problem



The Setup: Given an input sequence of 1, 2, and 3 arriving strictly in that order.

The Mechanism: We can interleave Push and Pop operations at will, but we must obey LIFO rules.

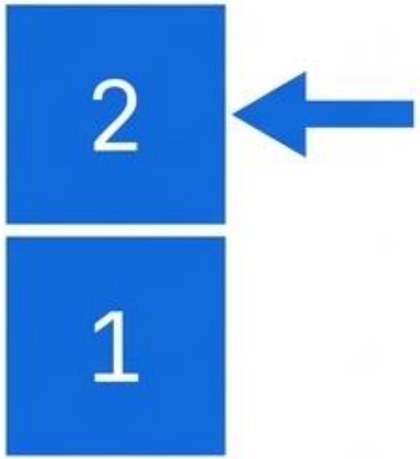
Valid Output Sequences: 123, 132, 213, 231, 321.

Example Trace for 213: Push 1, Push 2, Pop 2 (out), Pop 1 (out), Push 3, Pop 3 (out).

The Catch: Simple combinatorial math suggests 3! = 6 permutations. Why are there only 5 valid stack sequences?

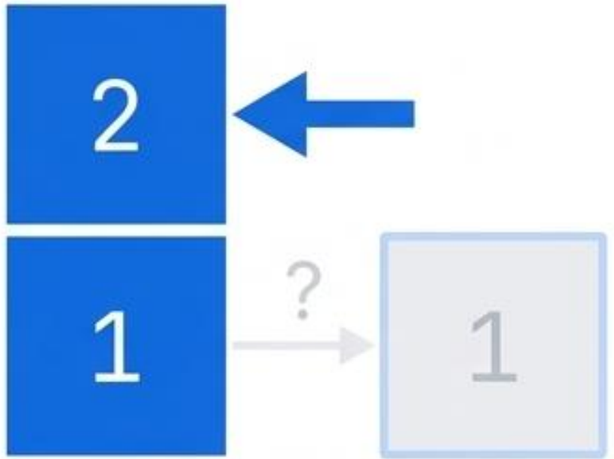
Visual Proof: The Impossibility of "3 1 2"

The State



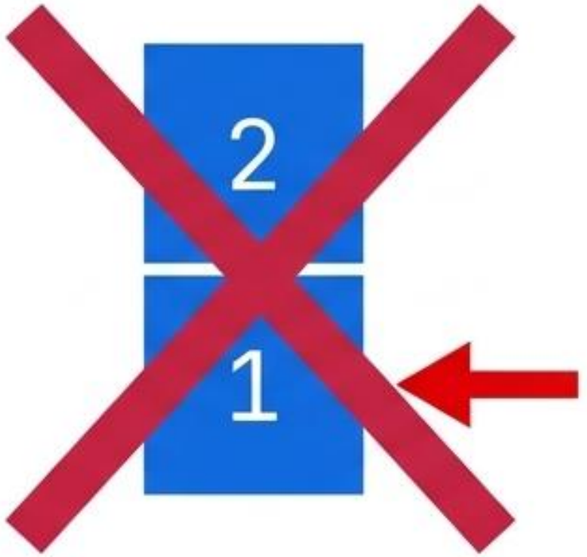
After 3 is popped, the remaining stack holds [1, 2]. The top pointer rests directly on 2.

The Blockade



The target sequence demands that 1 is the next output. However, LIFO architecture strictly dictates that 2 blocks access to 1.

Conclusion



Element 2 must be popped before 1. A sequence of 3 1 2 is mechanically impossible.

Computational Applications in the Wild



1. Expression Evaluation

Parsing mathematical expressions (Infix, Postfix, Prefix) and managing strict operation precedence in compilers.



2. Execution Call Stack

Storing return addresses, parameters, and local variables during deeply nested or recursive function calls in program execution.



3. State Reversal (Undo)

Managing chronological application states, where the most recently executed action is inherently the first state to be reversed.

The Stack: Architectural Summary

The Principle: A rigidly controlled data structure governed exclusively by the Last In, First Out (LIFO) mandate.

The Engine: Driven by a static 1-dimensional array and a single, dynamic integer top pointer executing in absolute O(1) time.

The Boundaries: Safely constrained between -1 (Underflow) and n-1 (Overflow).

The Legacy: A foundational computing primitive that trades arbitrary data access for unparalleled chronological predictability.